

RBE 500 – Homework 7

Tracy Yang

July 20, 2026

Problem 1: Robot Control

Part a) PD Gain Design

Plant: $J\ddot{\theta} + B\dot{\theta} = u$, with $J = 2$, $B = 0.4$.

PD control law (error $e = \theta_r - \theta$):

$$u = K_p e - K_d \dot{\theta}.$$

Closed loop:

$$J\ddot{\theta} + (B + K_d)\dot{\theta} + K_p\theta = K_p\theta_r \Rightarrow \ddot{\theta} + 2\zeta\omega_n\dot{\theta} + \omega_n^2\theta = \omega_n^2\theta_r,$$

$$2\zeta\omega_n = \frac{B + K_d}{J}, \quad \omega_n^2 = \frac{K_p}{J}.$$

Design specs: critically damped ($\zeta = 1$), $t_s = 4/\omega_n = 1$ s, so $\omega_n = 4$ rad/s.

$$K_p = J\omega_n^2 = 2(4)^2 = \mathbf{32}, \quad K_d = 2\zeta\omega_n J - B = 2(1)(4)(2) - 0.4 = \mathbf{15.6}$$

$$K_p = 32, \quad K_d = 15.6$$

Part b) Simulink Implementation

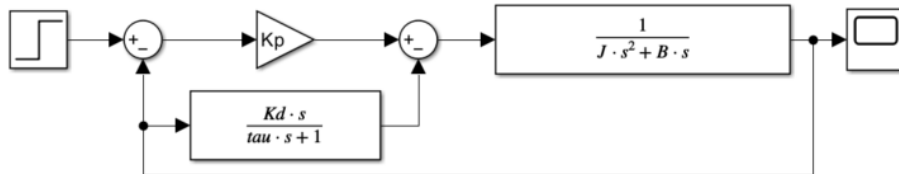


Figure 1: Closed-loop block diagram in Simulink (Problem 1).

Parameter script (params.m):

```
1 J = 2; % link inertia
2 B = 0.4; % effective damping
3 Kp = 32; % proportional gain
4 Kd = 15.6; % derivative gain
5 tau = 0.0001; % derivative filter time constant
```

Build notes:

- **Plant.** $J\ddot{\theta} + B\dot{\theta} = u$ transforms to $\Theta(s)/U(s) = 1/(Js^2 + Bs)$, torque to *position*, not velocity, so the plant has a free integrator. Transfer Fcn block: Numerator [1], Denominator [J B 0]. The trailing zero is required; [J B] alone gives torque to angular *velocity* instead of position.
- **Derivative term.** An ideal *Derivative* block on the error e fails two ways: it differentiates the Step block's jump at $t = 0$ (infinite slope, solver crashes), and it creates an **algebraic loop** with the plant, since both u and θ become instantaneous functions of each other. The fix is a *filtered derivative*: a second Transfer Fcn block realizing $\frac{K_d s}{\tau s + 1}$ (Numerator [Kd 0], Denominator [tau 1]), tapped off the plant output θ rather than the error. This block has its own internal state, so it cannot form an algebraic loop, and it rolls off at high frequency instead of differentiating a discontinuity exactly. It is the same technique Simulink's built-in PID block uses internally.

Numerical verification. To avoid misreading settling time and overshoot off the scope, the closed-loop response was also computed directly from the transfer functions:

```
1 J = 2; B = 0.4; Kp = 32; Kd = 15.6; tau = 0.0001;
2
3 P = tf(1, [J B 0]); P.InputName = 'u'; P.OutputName = 'theta';
4 Dfilt = tf([Kd 0], [tau 1]); Dfilt.InputName = 'theta'; Dfilt.OutputName = 'ud';
5 Kp_tf = tf(Kp, 1); Kp_tf.InputName = 'e'; Kp_tf.OutputName = 'up';
6
7 sum1 = sumblk('e = xr - theta');
8 sum2 = sumblk('u = up - ud');
9 CL = connect(P, Dfilt, Kp_tf, sum1, sum2, 'xr', 'theta');
10
11 t = 0:0.0001:3;
12 y = step(CL, t);
13 info = stepinfo(y, t)
```

```
info =

struct with fields:

    RiseTime: 0.8394
    TransientTime: 1.4579
    SettlingTime: 1.4579
    SettlingMin: 0.9000
    SettlingMax: 0.9999
    Overshoot: 0
    Undershoot: 0
    Peak: 0.9999
    PeakTime: 3
```

$t_s \approx 1.46$ s with 0% overshoot, slower than the 1 s target. This is addressed in Part c.

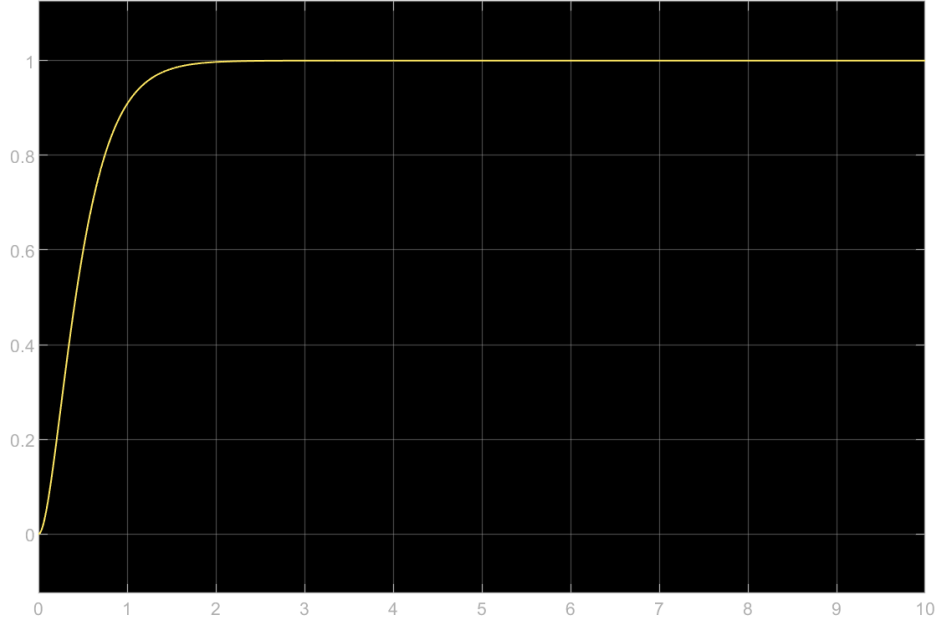


Figure 2: Step response, $K_p = 32$, $K_d = 15.6$.

Part c) Tuning

Why t_s exceeds 1 s. The formula $t_s = 4/(\zeta\omega_n)$ comes from the decaying exponential envelope of an underdamped response. The exact critically-damped response is

$$\theta(t) = 1 - (1 + \omega_n t)e^{-\omega_n t},$$

which decays slower than the plain exponential envelope, so at $\zeta = 1$ the formula underestimates t_s . This matches the measured 1.46 s versus the 1 s target.

A modeling bug found along the way. A mismatch between Simulink and MATLAB results (predicted $t_s \approx 0.4$ s, observed $t_s \approx 3$ s) was traced by setting $K_d = 0$: the resulting slow, lightly-damped oscillation confirmed the controller logic was correct but pointed at the plant. The Transfer Fcn had been entered as $[J \ B]$, missing the free integrator; correcting it to $[J \ B \ 0]$ resolved the mismatch.

Retuning. Requiring $(1 + \omega_n)e^{-\omega_n} \leq 0.02$ for $t_s = 1$ s gives $\omega_n \gtrsim 6$ numerically; $\omega_n = 6.5$ was used for margin:

$$K_p = J\omega_n^2 = 2(6.5)^2 = 84.5, \quad K_d = 2\omega_n J - B = 2(6.5)(2) - 0.4 = 25.6$$

```

1 J = 2; B = 0.4; Kp = 84.5; Kd = 25.6; tau = 0.0001;
2 % same connect() structure as above
3 t = 0:0.0001:3;

```

```

4 y = step(CL, t);
5 info = stepinfo(y, t)

```

```

info =

    struct with fields:

        RiseTime: 0.5168
        TransientTime: 0.8981
        SettlingTime: 0.8981
        SettlingMin: 0.9000
        SettlingMax: 1.0000
        Overshoot: 0
        Undershoot: 0
        Peak: 1.0000
        PeakTime: 3

```

$$K_p = 84.5, \quad K_d = 25.6, \quad \tau = 0.0001, \quad t_s \approx 0.898 \text{ s}$$

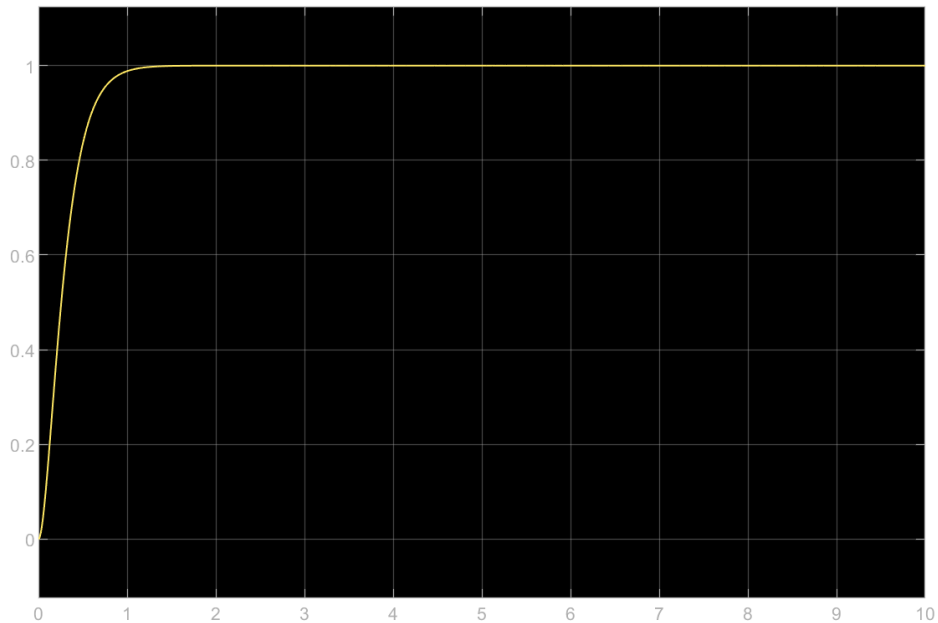


Figure 3: Step response after tuning.

Problem 2: Robot Control

System: $m\ddot{x} + b\dot{x} + kx = F$, with $m = 1$, $k = 1$, $b = 0.5$.

PD law: $F = K_p(x_r - x) - K_d\dot{x}$.

Closed loop:

$$m\ddot{x} + (b + K_d)\dot{x} + (k + K_p)x = K_p x_r \Rightarrow 2\zeta\omega_n = b + K_d, \quad \omega_n^2 = k + K_p.$$

Design specs: $t_s = 2$ s, no overshoot ($\zeta = 1$), so $\omega_n = 4/2 = 2$ rad/s.

$$K_p = \omega_n^2 - k = 4 - 1 = 3, \quad K_d = 2\omega_n - b = 4 - 0.5 = 3.5$$

$$K_p = 3, \quad K_d = 3.5 \quad (\text{initial design; corrected below})$$

This plant already has a restoring term k , so $1/(ms^2 + bs + k)$ is genuinely second order. Transfer Fcn: Numerator [1], Denominator [m b k], with no trailing zero needed.

Steady-State Error

Checking $K_p = 3$, $K_d = 3.5$ numerically:

```
1 m = 1; k = 1; b = 0.5; Kp = 3; Kd = 3.5; tau = 0.0001;
2
3 P = tf(1, [m b k]); P.InputName = 'F'; P.OutputName = 'x';
4 Dfilt = tf([Kd 0], [tau 1]); Dfilt.InputName = 'x'; Dfilt.OutputName = 'ud';
5 Kp_tf = tf(Kp, 1); Kp_tf.InputName = 'e'; Kp_tf.OutputName = 'up';
6
7 sum1 = sumblk('e = xr - x');
8 sum2 = sumblk('F = up - ud');
9 CL = connect(P, Dfilt, Kp_tf, sum1, sum2, 'xr', 'x');
10
11 t = 0:0.0001:5;
12 y = step(CL, t);
13 info = stepinfo(y, t)
```

info =

struct with fields:

```
    RiseTime: 1.6763
TransientTime: 2.9032
SettlingTime: 2.9032
SettlingMin: 0.6747
SettlingMax: 0.7496
    Overshoot: 0
    Undershoot: 0
        Peak: 0.7496
    PeakTime: 5
```

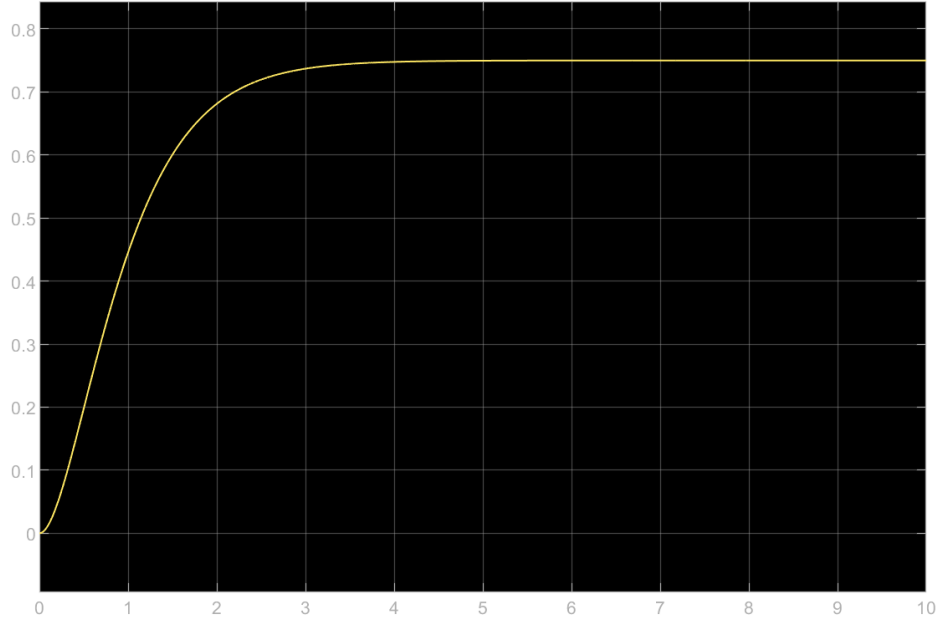


Figure 4: Step response with $K_p = 3$, $K_d = 3.5$, no reference scaling. The output settles at 0.75, not 1.

The output settles at 0.75, not 1. At steady state ($\dot{x} = \ddot{x} = 0$), the closed-loop equation gives

$$x_{ss} = \frac{K_p}{k + K_p} x_r = \frac{3}{4} = 0.75.$$

The spring constantly pulls the mass back toward $x = 0$, and pure P control only balances it proportionally; it never fully cancels the spring's restoring force. Problem 1 had no such term, so its steady-state gain was always exactly 1.

Fix: reference scaling. A gain N is inserted between the Step block and the first Sum block:

$$N = \frac{k + K_p}{K_p} \Rightarrow x_{ss} = \frac{K_p}{k + K_p} (N x_r) = x_r.$$

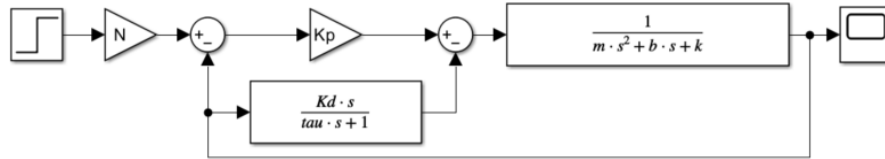


Figure 5: Closed-loop block diagram in Simulink (Problem 2), with reference-scaling gain N .

Retuning

With N added, $K_p = 3$, $K_d = 3.5$ gives $t_s \approx 2.90$ s, still past the 2 s target for the same reason discussed in Part 1c. Requiring $\omega_n t_s \gtrsim 6$ for a 2 s target gives $\omega_n \gtrsim 3$; $\omega_n = 3.5$ was used for margin:

$$K_p = \omega_n^2 - k = (3.5)^2 - 1 = 11.25, \quad K_d = 2\omega_n - b = 7 - 0.5 = 6.5, \quad N = \frac{k + K_p}{K_p} = 1.0889$$

```
1 m = 1; k = 1; b = 0.5; Kp = 11.25; Kd = 6.5; tau = 0.0001;
2 N = (k + Kp) / Kp;
3
4 P = tf(1, [m b k]); P.InputName = 'F'; P.OutputName = 'x';
5 Dfilt = tf([Kd 0], [tau 1]); Dfilt.InputName = 'x'; Dfilt.OutputName = 'ud';
6 Kp_tf = tf(Kp, 1); Kp_tf.InputName = 'e'; Kp_tf.OutputName = 'up';
7 Nref = tf(N, 1); Nref.InputName = 'xr'; Nref.OutputName = 'xr_scaled';
8
9 sum1 = sumblk('e = xr_scaled - x');
10 sum2 = sumblk('F = up - ud');
11 CL = connect(P, Dfilt, Kp_tf, Nref, sum1, sum2, 'xr', 'x');
12
13 t = 0:0.0001:5;
14 y = step(CL, t);
15 info = stepinfo(y, t)
```

```
info =

struct with fields:

    RiseTime: 0.9595
    TransientTime: 1.6673
    SettlingTime: 1.6673
    SettlingMin: 0.9000
    SettlingMax: 1.0000
    Overshoot: 0
    Undershoot: 0
    Peak: 1.0000
    PeakTime: 5
```

$$K_p = 11.25, \quad K_d = 6.5, \quad N = 1.0889, \quad t_s \approx 1.667 \text{ s}$$

Settles exactly at $x = 1$, inside the 2 s target, with 0% overshoot.

Parameter script (params2.m):

```
1 m = 1; % mass
```

```

2 k = 1; % spring constant
3 b = 0.5; % damping coefficient
4 Kp = 11.25; % proportional gain
5 Kd = 6.5; % derivative gain
6 N = (k+Kp)/Kp; % reference scaling
7 tau = 0.0001; % derivative filter time constant

```

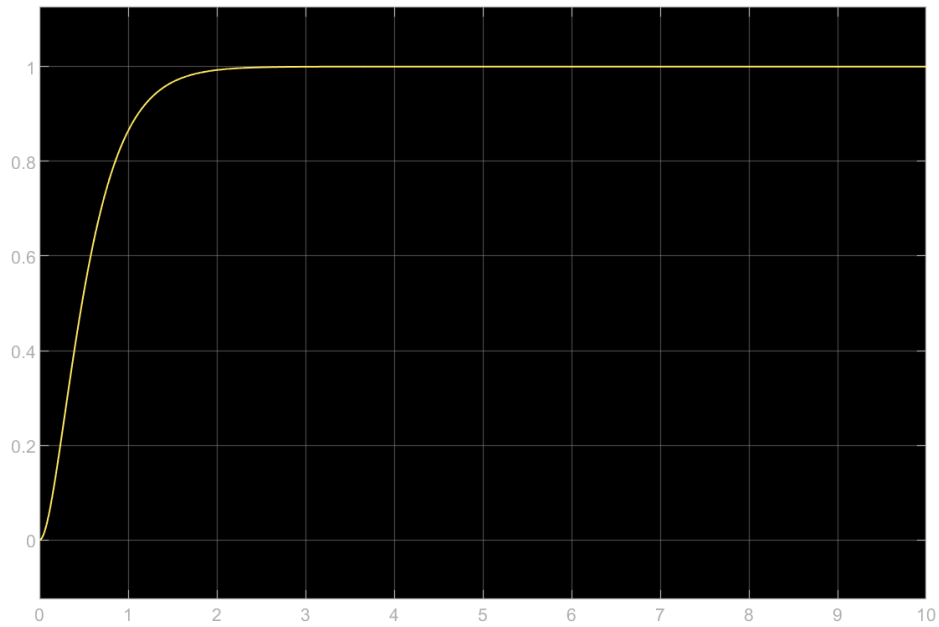


Figure 6: Step response, $K_p = 11.25$, $K_d = 6.5$, $N = 1.0889$.